SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques.*
(BL : 3)

Assessment Tool Weightage: 10%

QUESTIONS: 20 **Total Marks:20**
Duration :1 Hour

Mock Interview question

Code of Conduct:
Shaik Hafeeza Tabassum
I 192372014 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Hafeez

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?
2. Define the components of a PDA formally.
3. What is a stack in PDA? Why is it important?
4. What is an Instantaneous Description (ID) in PDA?
5. What are the different types of moves in a PDA?
6. What is acceptance by final state and empty stack?
7. Define the language accepted by a PDA.
8. How does a PDA process a string using IDs?
9. Differentiate between Deterministic PDA and Non-Deterministic PDA.
10. Why are PDAs equivalent to Context-Free Grammars (CFG)?
11. What types of languages are accepted by PDA? Give examples.
12. What is a Turing Machine? How is it more powerful than PDA?
13. Define the components of a Turing Machine.
14. Explain tape, head, and state transitions in a Turing Machine.
15. Explain programming techniques for Turing Machines.
16. What is storage in finite control in Turing Machines?
17. Compare FA, PDA, and TM based on memory, power, and language acceptance.
18. What is DFA?
19. Which language is accepted by a Turing Machine?
20. How turing machines are applied in real world applications?

RUBRICS FOR CALCULATION

1. Pushdown Automata:

Pushdown automata (PDA) is a computational model that uses a stack along with finite states to recognize context-free-grammar languages (CFLs)

Difference:

⇒ Finite Automata has only finite states and no extra memory

⇒ PDA has a stack for storing information

⇒ PDA can be recognize languages such as $\{a^n b^n | n \geq 0\}$, which FA cannot recognize.

2. Define the component of PDA?

A PDA is formally represented as a 7-tuple:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where,

- Q - Set of states
- Σ - Input alphabet
- Γ - Stack alphabet
- δ - Transition Function
- q_0 - Initial state
- Z_0 - Initial stack symbol
- F - Set of final states

(3) What is a stack in PDA? Why it is important?

A stack is a memory structure used by PDA that follows LIFO (Last in First out).

It supports:

- push – Add symbol to stack
- pop – Remove symbol from stack
- Read/Inspect – Check the top symbol.

The stack is important because it allows a PDA to remember an unbounded amount of information, helping it recognizes languages such as balanced parantheses and $\{a^n b^n\}$

(4) What is an Instantaneous Description (ID) in PDA?

An Instantaneous Description (ID) represents the Concurrent configuration of a PDA

It is written as:

(q, w, γ)

where:

q – current state

w = remaining input

γ = current stack contents

It shows the complete status of PDA at a particular moment

(5) What are the different types of moves in PDA?

The main types of PDA moves are,

1. Input-consuming Move
2. ε-move
3. push move
4. pop move

) What is Acceptance by final state and empty stack?

Those are two methods of accepting a string in PDA:

1. Acceptance by Final State:

The string is accepted when the PDA consumes the input and reaches the Final State

2. Acceptance by Empty Stack:

The string is accepted when the PDA consumes the input and stack becomes empty

⇒ Both methods are used to define CFL's

7) Define the language accepted by PDA?

The language accepted by PDA is set of all strings that PDA accepts according to its acceptance condition.

For a PDA M:

$$L(M) = \{w \in \Sigma^* \mid w \text{ accepted by } M\}$$

PDA's generally accept context-Free languages (CFL)

8. How does a PDA process a string using IDS?

PDA processes a string through a sequence of Instantaneous Descriptions (IDS)

Ex: $(q_0, w, z_0) \vdash (q_1, w', Az_0) \vdash \dots \vdash (q_f, \varepsilon, v)$

(9) Difference between DPDA and NPDA

DPDA	NPDA
• Has only one possible move for configuration	• Can have multiple possible moves
• Deterministic	• Non-determinity
• Less powerful	• More powerful
• Recognizes D-CFLs	• Recognises all CFLs
• Cannot Recognize Every CFL	• Can Recognize Every CFL

(10) Why PDA's are Equivalent to CFGs?

PDAs are Equivalent to CFG because they describe the same class of Languages.

CFG → PDA ⇒ Can be simulate the derivations of CFG

PDA → CFG ⇒ Can be generate strings accepted by PDA

∴ Languages generated by CFGs = Language accepted by NPDA = Context Free Languages.

11. What types of languages are accepted by PDA? Give Examples?

A PDA accepts Context-Free languages (CFLs)

Ex:
- $L = \{a^n b^n \mid n \geq 0\}$
- Balanced parantheses
- Properly nested structures
- Palindromes such as

12. What is the turing machine? How is it more powerful than PDA?

Turing machine (TM) is a mathematical model of computation that uses an unbounded tape, a read/write head, and a finite set of states.

⇒ It is more powerful than a PDA:
- PDA has a stack with limited access.
- TM can be read and write anywhere on its tape
- TM can move its head left (or) right

(13) Define the components of a Turing machine?

A turing machine can be represented as a 7-tuple

$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$

where: Q - set of state
Σ - Input alphabet
δ - Transition Function
Γ - Tape alphabet
q_0 - Initial state
B - Blank symbol
F - set of Final states

(14) Explain Tape, Head, and state transition in a Turing machine?

- Tape: Stores input and working symbols. It is unbounded

- Head: Read and writes symbols on the tape and moves Left (L) or Right (R).
- State Transition: Determines the next state, symbol to write, and head movement.

A transition can be represented as:

$\delta(q, a) = (p, b, R)$ → move right
(state, reading, write)

(15) Explain programming techniques for Turing machines?

1. Storing information in states
2. Using markers on the tape
3. Moving left and right to compare symbols
4. Using subroutines for repeated operations
5. Shifting or copying symbols on tape

(16) What is Storage in finite control in Turing Machines?

⇒ Finite control stores a ~~limited~~ amount of information in the states of a Turing machine.

For example, a state can remember:

- which symbol was previously encountered
- which stage of computation is currently running

(17) Compare FA, PDA, and TM based on memory, power and language acceptance?

Feature	FA
Memory	Finite states
Power	Lowest
Accepted Languages	Regular languages
Memory access	No Extra memory
Example	a^*b^*

18. What is DFA?

Deterministic Finite Automata (DFA) is a Finite automata machine in which, for every state and input symbol, there is exactly one transition.

It is represented by

$$D = (Q, \Sigma, \delta, q_0, F).$$

DFA accepts Regular Languages

19. Which Language is accepted by a Turing Machine?

A Turing machine can accept Recursively Enumerable (RE) Languages.

Therefore, Turing machines are more powerful and than Finite Automata and pushdown Automata

20. How are Turing machines Applied in real-world Applications?

Turing machines are mainly theoretical Models, but their concepts form the foundation of Modern Computing

Application Include:

- Designing and analysing Algorithms
- understanding computer program computation
- Compiler and programing language theory
- Modeling general-purpose computers.